

Software Development Fundamentals (SDF) – I

ODD 2020



WEEK 11
LECTURE 31 - 35

Structures and Union

Jaypee Institute of Information Technology (JIIT)
A 10, Sector 62, Noida

SDF-1: Lecture Plan for Module 5(4 lectures)

Lecture No.	Topics to be Discussed
31	Introduction and implementation of structure
32	Union in C programming
33	Array of structures and related operation like insertion, traversal, updating etc. in C programming using different problems
34 & 35	Structure using function

Outline: Lecture 31

- Why use Structure
- How to declare/ define a structure
- How to access member of structure
- How structure elements stored

Data Types

There are various datatypes available in C programming language

- Basic Data type
 - Integer, Char, float, void
- Structured Data type
 - Array, Strings
- User Defined Data type
 - Enum, **Structures, Unions**

Structures

- A **Structure** is a collection of heterogeneous data types.
- **struct** keyword is used to create a structure .
- Structure is collections of different data types
In contrast, **array** is collection of similar data type.

Structures Contd..

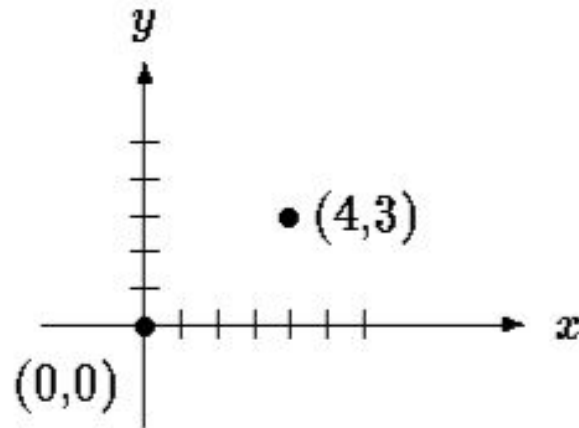
- All the members of a structure are stored at **contiguous memory locations**.
- A **variable** of structure type can store **multiple data items** of different data types under the one name.
- As the data of employee in an organization that is ename, eID, esalary, eaddress, ephone number is stored in structure data type.

Structures

- Structure is a user-defined data type in C language which allows us to combine data of different types together.
- Examples:
 - Student record: student id, name, major, gender, start year, ...
 - Bank account: account number, name, currency, balance, ...
 - Address book: name, address, telephone number, ...
 - Employee record: employee id, e_name, e_mob, e_address, e_salary,...
- In database applications, structures are called records.

Example :

Let us create a few structures suitable for graphics. The basic object is a point, which we will assume has an x coordinate and a y coordinate, both integers.



The two components can be placed in a structure declared like this:

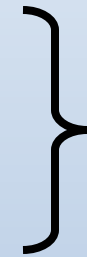
```
struct point {  
    int x;  
    int y;  
};
```

The keyword `struct` introduces a structure declaration, which is a list of declarations enclosed in braces. An optional name called a *structure tag* may follow the word `struct` (as with `point` here). The tag names this kind of structure, and can be used subsequently as a shorthand for the part of the declaration in braces.

How to define structure?

- Definition of a structure:

```
struct <struct-tag>{  
    <type> <identifier_list>;  
    <type> <identifier_list>;  
    . . .  
} ;
```



Each identifier
defines a
member
of the
structure.

- Example:

```
struct Date {  
    int day;  
    int month;  
    int year;  
} ;
```



The “Date” structure
has 3 members,
day, month & year.

struct examples

- Example:

```
struct StudentData{  
    int S_Id;  
    int S_age;  
    char S_Gender;  
    double S_CGA;  
};
```



The “StudentData” structure has 4 members of different types.

- Example:

```
struct StudentGrade{  
    char Name[15];  
    char Course[9];  
    int Lab[5];  
    int Assignment[3];  
    int Exam[2];  
};
```



The “StudentGrade” structure has 5 members of different array types.

struct examples

- Example:

```
struct EmployeeRecord{  
    char Name[15];  
    int Id;  
    char Dept[5];  
    char Gender;  
    float Salary;  
};
```



The “EmployeeRecord”
structure has 5
members.

Declaring a Structure Variable

The syntax of declaring a structure variable is

struct <struct_name> <variable_name>;

The example to declare the variable for defined structure “EmployeeRecord”

struct EmployeeRecord e1;

Here **e1 variable contains 5** members that are defined in structure.

Accessing a Structure Members

- Members of the structure can not be accessed directly.
- They are accessed by using the name of structure variable followed by a dot and then the name of member variable.

```
struct EmployeeRecord{  
    char Name[15];  
    int Id;  
    char Dept[5];  
    char Gender;  
    float Salary;  
};
```

```
struct EmployeeRecord e1;
```

- The method used to access the structure variables are **e1.Name, e1.Id, e1.Dept, e1.Gender, e1.Salary.**
- The data with in the structure is stored and printed by this method using scanf and printf statement in c program.

Initializing a Structure Members

The members of individual structure variable can be initialized one by one or in a single statement.

The example to initialize a structure variable is

1) `struct EmployeeRecord e1 = {"Arnab",101, "CBI", 'M', 100000.00};`

2) `e1.Name="Arnab";`
`e1.Id= 101;`
`e1.Dept="CBI";`
`e1.Gender='M';`
`e1.Salary=100000.00;`

```
struct EmployeeRecord{  
    char Name[15];  
    int Id;  
    char Dept[5];  
    char Gender;  
    float Salary;  
};
```

Structure Types

- ♦ Arrays are collections of related data of the same data type (homogeneous).
- ♦ Structures are collections of related data of possibly different data types (heterogeneous.)
- ♦ There are a few ways to define a structure.

Structure Types (Cont.)

- ◆ One method:

```
struct struct_tag {  
    type1 id_list1;  
    type2 id_list2;  
    ...  
};
```

- ◆ Examples:

```
struct fract {  
    int num, den;  
};
```

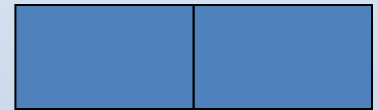
```
struct personal {  
    char name[20];  
    char gender;  
    int age;  
    float height;  
};
```


Structure Types (cont.)

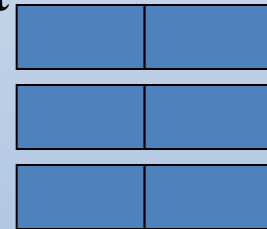
- ♦ To declare variables of these structures:

```
struct fract my_fract, list[3];
```

my_fract



list



list[0]

list[1]

list[2]

```
struct personal ob, group[20];
```

Structure Types (cont.)

- Second method: combine definition of a structure type and declaration of variables of that type together.

```
struct fract {  
    int num, den;  
};
```

+

```
struct fract my_fract,  
                list[3];
```

=

```
struct fract {  
    int num, den;  
} my_fract, list[3];
```

Tag is optional here.



Some important points

1. **struct** keyword is used to declare structure.
2. Members of structure are enclosed within opening and closing braces.
3. There is no memory space occupied when structure is declared .
4. Memory is created, very first time when the variable is created.

Structure

Type	Using normal variable
Syntax	<pre>struct tag_name { data type var_name1; data type var_name2; data type var_name3; };</pre>
Example	<pre>struct student { int mark; char name[10]; float average; };</pre>
Declaring structure variable	<pre>struct student report;</pre>
Initializing structure variable	<pre>struct student report = {100, "Mani", 99.5};</pre>
Accessing structure members	<pre>report.mark report.name report.average</pre>

Limitations of C Structures

In C language, Structures provide a method for packing together data of different types. A Structure is a helpful tool to handle a group of logically related data items. However, C structures have some limitations.

- The C structure does not allow the struct data type to be treated like built-in data types.
- We cannot use operators like +,- etc. on Structure variables.
- **No Data Hiding:** C Structures do not permit data hiding. Structure members can be accessed by any function, anywhere in the scope of the Structure.
- **Functions inside Structure:** C structures do not permit functions inside Structure
- **Static Members:** C Structures cannot have static members inside their body
- **Access Modifiers:** C Programming language do not support access modifiers. So they cannot be used in C Structures.

Following program to demonstrate the accessing the structure member:

```
main( )
{
    struct book
    {
        char name ;
        float price ;
        int pages ;
    };
    struct book b1, b2, b3 ;

    printf ( "\nEnter names, prices & no. of pages of 3 books\n" ) ;
    scanf ( "%c %f %d", &b1.name, &b1.price, &b1.pages ) ;
    scanf ( "%c %f %d", &b2.name, &b2.price, &b2.pages ) ;
    scanf ( "%c %f %d", &b3.name, &b3.price, &b3.pages ) ;

    printf ( "\nAnd this is what you entered" ) ;
    printf ( "\n%c %f %d", b1.name, b1.price, b1.pages ) ;
    printf ( "\n%c %f %d", b2.name, b2.price, b2.pages ) ;
    printf ( "\n%c %f %d", b3.name, b3.price, b3.pages ) ;
}
```

And here is the output...

```
Enter names, prices and no. of pages of 3 books
A 100.00 354
C 256.50 682
F 233.70 512
```

How Structure Elements are stored

Whatever be the elements of a structure, they are always stored in contiguous memory locations. The following program would illustrate this:

```
/* Memory map of structure elements */
main( )
{
    struct book
    {
        char name ;
        float price ;
        int pages ;
    };
    struct book b1 = { 'B', 130.00, 550 };

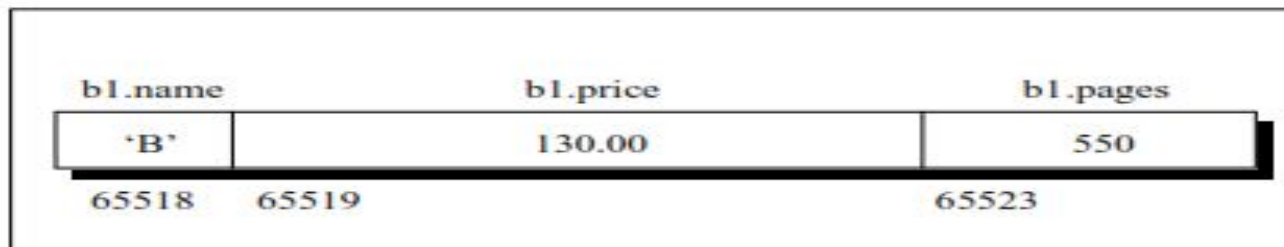
    printf ( "\nAddress of name = %u", &b1.name );

    printf ( "\nAddress of price = %u", &b1.price );
    printf ( "\nAddress of pages = %u", &b1.pages );
}
```

Here is the output of the program...

Address of name = 65518
Address of price = 65519
Address of pages = 65523

Actually the structure elements are stored in memory as shown in the Figure 14.14

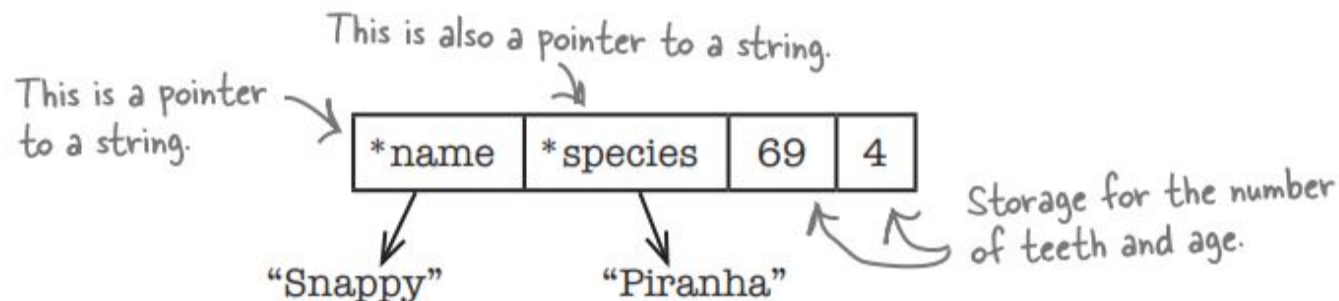


When you define a `struct`, you're not telling the computer to create anything in memory. You're just giving it a **template** for how you want a new type of data to look.

```
struct fish {  
    const char *name;  
    const char *species;  
    int teeth;  
    int age;  
};
```

But when you define a new variable, the computer will need to create some space in memory for an **instance** of the `struct`. That space in memory will need to be big enough to contain all of the fields within the `struct`:

```
struct fish snappy = {"Snappy", "Piranha", 69, 4};
```



- Write a C program to add two distances entered by user using structure.
- Measurement of distance should be in inch and feet.(Note: 12 inches = 1 foot)

```

#include <stdio.h>

struct Distance {
    int feet;
    float inch;
} d1, d2, result;

int main() {
    // take first distance input
    printf("Enter 1st distance\n");
    printf("Enter feet: ");
    scanf("%d", &d1.feet);
    printf("Enter inch: ");
    scanf("%f", &d1.inch);

    // take second distance input
    printf("\nEnter 2nd distance\n");
    printf("Enter feet: ");
    scanf("%d", &d2.feet);
    printf("Enter inch: ");
    scanf("%f", &d2.inch);

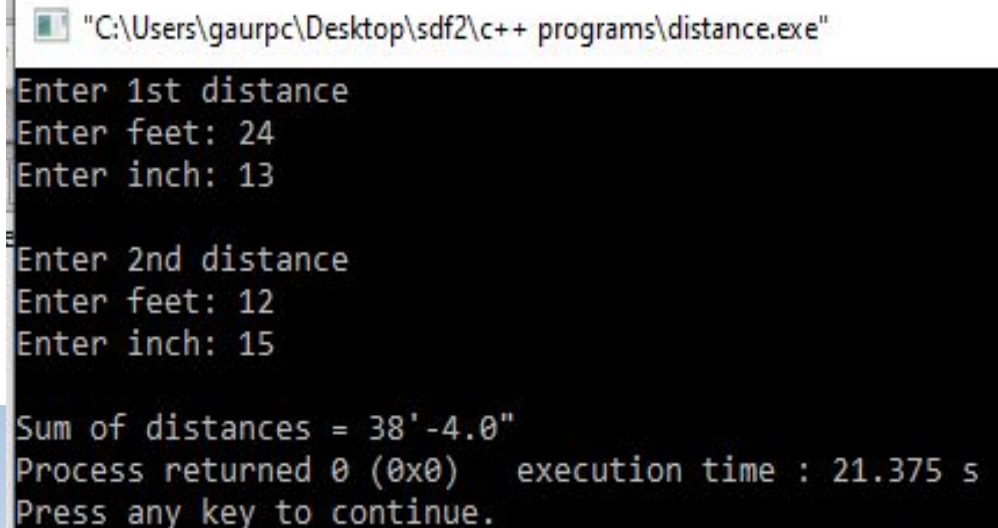
```

```

// adding distances
result.feet = d1.feet + d2.feet;
result.inch = d1.inch + d2.inch;

// convert inches to feet if greater than 12
while (result.inch >= 12.0) {
    result.inch = result.inch - 12.0;
    ++result.feet;
}
printf("\nSum of distances = %d\'-%.1f\"", result.feet, result.inch);
return 0;

```



```

"C:\Users\gaurpc\Desktop\sdf2\c++ programs\distance.exe"
Enter 1st distance
Enter feet: 24
Enter inch: 13

Enter 2nd distance
Enter feet: 12
Enter inch: 15

Sum of distances = 38'-4.0"
Process returned 0 (0x0)   execution time : 21.375 s
Press any key to continue.

```

Exercise

Write a program that compares two given dates. To store date use structure say **date** that contains three members namely date, month and year. If the dates are equal then display message as "Equal" otherwise "Unequal".

End of Lecture 31

Outline: Lecture 32

- Why use Typedef
- How to declare/ define a Union
- How to access member of Union
- How union elements stored

Using Typedef

Using typedef

- ◆ For creating **synonyms** (**aliases**) for previously defined data types.
- ◆ The typedef keyword defines a new name for an existing type.
- ◆ Examples:

```
typedef int integ;  
typedef char charac;
```

- ◆ **integ** is now a synonym for type **int**;
- ◆ **charac** is a synonym for type **char**; and
- ◆ Now we can write:

```
integ a, b, c; /* a, b, c are of type int */  
charac v;
```

Typedef Contd..

```
typedef struct {  
    int num, den;  
} Fract;  
Fract my_fract, list[3];
```

Advantage: No need to mention 'struct' keyword while declaring structure variables now

How to declare structure using typedef in C:

- **typedef struct** Ganesha
{
 char type [10];
 float height;
 char colour[20];
} gift;
- To declare structure variable, we can use the below statements.
- gift ekdanta; /*ekdanta is structure variable */
- gift dvimukha; /*dviukha is structure variable */

Example

```
// Structure using typedef:
```

```
#include <stdio.h>
#include <string.h>
```

```
typedef struct student
{
    int id;
    char name[20];
    float percentage;
} status;
```

```
int main()
{
    status record;
    record.id=1;
    strcpy(record.name, "ABC");
    record.percentage = 86.5;
    printf(" Id is: %d \n", record.id);
    printf(" Name is: %s \n", record.name);
    printf(" Percentage is: %f \n", record.percentage);
    return 0;
}
```

Output

```
Id is: 1
Name is: ABC
Percentage is: 86.500000
```

Output of program?

```
#include<stdio.h>
void main()
{
    struct{
        int ssn;
        int pages;
    }book;

    book.ssn=101;
    book.pages=350;
    printf("\nSSN of book: %d", book.ssn);
    printf("\nPages in Book: %d\n",book.pages);
}
```

Output of program?

```
#include<stdio.h>
void main()
{
    struct{
        int ssn;
        int pages;
    }book;

    book.ssn=101;
    book.pages=350;
    printf("\nSSN of book: %d", book.ssn);
    printf("\nPages in Book: %d\n",book.pages);
}
```

SSN of book: 101
Pages in Book: 350

UNION

Unions

A *union* is a memory location that is shared by two or more different types of variables. A union provides a way of interpreting the same bit pattern in two or more different ways. Declaring a **union** is similar to declaring a structure. Its general form is

```
union tag {  
    type member-name;  
    type member-name;  
    — type member-name;  
    .  
    .  
    .  
} union-variables;
```

For example:

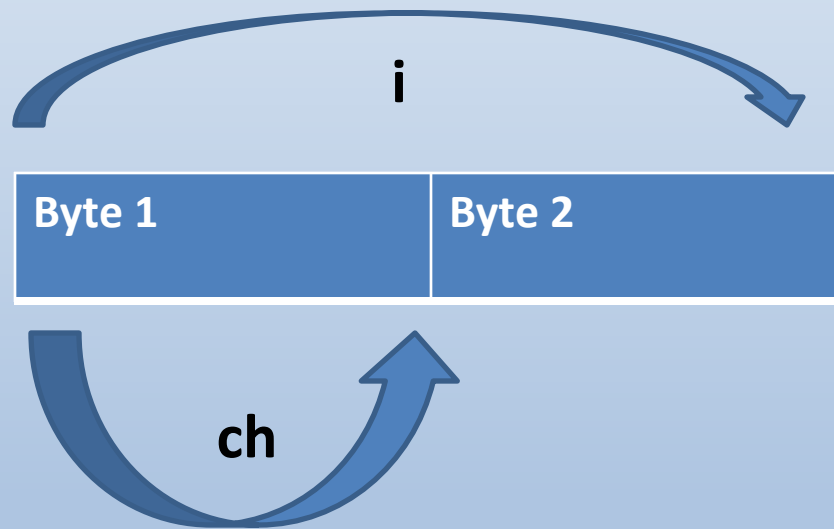
```
union u_type {  
    int i;  
    char ch;  
};
```

```
union abc {  
    int i;  
    char ch;  
};
```

- Above declaration does not create any variables.
- We can declare a variable either by placing its name at the end of the declaration or by using a separate declaration statement.
- To declare a union variable called uob of type abc using the definition just given, write

```
union abc uob;
```

- In uob, both integer i and character ch share the same memory location.
- Assuming 2byte for integer and 1 byte for character, how i and ch share the same address.



- To access a member of a union, use the same syntax that we used for structures: the dot and arrow operators.
- If you are operating on the union directly, use the dot operator. If the union is accessed through a pointer, use the arrow operator.
- For example, to assign the integer 30 to element i of uob and assign character 'M' to ch.

uob.i=30;

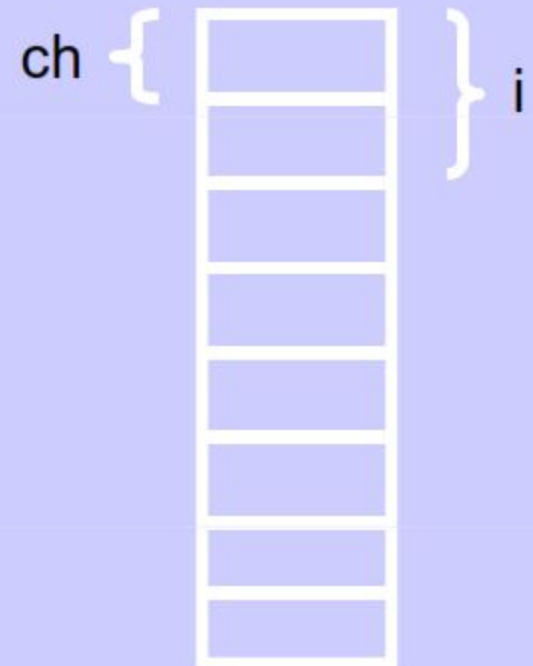
uob.ch='M';

Structure Vs Union

```
struct number{  
    char ch;  
    int i;  
} x;
```



```
union number{  
    char ch;  
    int i;  
} y;
```



Structure Vs Union

- Members of a union can only be accessed one at a time

```
x.i = 100;  
x.ch = 'a';  
printf("%c,%d",x.ch,x.i);
```

Output:
a,100

```
y.i = 100;  
y.ch = 'a';  
printf("%c,%d",y.ch,y.i);
```

Output:
a,1627390052

- Since union members have the same memory address, changing the value of one alters the value of the other

Example:

A union may be member of a structure, and a structure may be a member of a union

```
union id {
```

```
char color[12];
```

```
int size;
```

```
};
```

```
struct clothes{
```

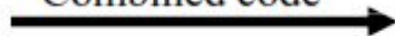
```
char manufacturer[20];
```

```
float cost;
```

```
union id description;
```

```
} shirt;
```

Combined code



```
struct clothes{
```

```
char manufacturer[20];
```

```
float cost;
```

```
union {
```

```
char color[20];
```

```
int size;
```

```
}description;
```

```
} shirt;
```

Why Do We Need Unions?

- Suppose there are three types of products: books, mugs, and shirts.
 - Books: Title, author, number of pages
 - Mugs: Design
 - Shirts: color, size
- We want to store all the items in an array. So, we must use the same data type to describe all three different kinds of products

```
struct item{  
    int stock_number;  
    float price;  
    int item_type;  
    union{  
        struct{  
            char title[100];  
            char author[100];  
            int num_pages;  
        } book;  
    };  
};
```

```
struct{  
    char design[100];  
} mug;  
struct {  
    int size;  
    char color[20];  
} shirt;  
} item;  
};
```

```

#include<stdio.h>
#include<stdlib.h>
struct item
{
    int stock_number;
    float price;
    int item_type;
    union{
        struct{
            char title[100];
            char author[100];
            int num_pages;
        }book;
        struct{
            char design[100];
        }mug;
        struct{
            char size;
            char color[20];
        }shirt;
    }item;
};

```

```

void main()
{
    int j;
    struct item x;
    printf("\nenter gift type \t 0: book \t 1: Mug \t 2:
shirt\n");
    scanf("%d",&x.item_type);
    fflush(stdin);
    switch(x.item_type)
    {
        case 0: printf("\n enter book title ");
                gets(x.item.book.title);
                printf("\nthe title of the book is  ");
                puts(x.item.book.title);
                break;
        case 1: printf("\n enter mug design ");
                scanf("%s",x.item.mug.design);
                printf("\ndesign of mug is  ");
                puts(x.item.mug.design);
                break;
        case 2: printf("\nenter size and colour of shirt ");
                scanf("%c", & x.item.shirt.size);
                scanf("%s", x.item.shirt.color);
                printf("\nsize of shirt is %c and colour is
%s",x.item.shirt.size,x.item.shirt.color);
                break;
    }
}

```

output

```
enter gift type          0: book          1: Mug          2: shirt
0

  enter book title letusC

the title of the book is  letusC
```

```
enter gift type          0: book          1: Mug          2: shirt
1

  enter mug design coffeemug

design of mug is  coffeemug
```

```
enter gift type          0: book          1: Mug          2: shirt
2

enter size and colour of shirt M yellow

size of shirt is M and colour is yellow
```

End of Lecture 32

Outline: Lecture 33

- Array of structures and related operation like insertion, traversal, updating etc. in C programming using different problems.

Array of Structure

- An array is a collection of similar data types. Similarly we can also define array of structure.
- The array of structure is used to store the **large number of similar records**.
- For example to store the record of 100 students then **array of structure is used**.
- The method to define and access the array element of array of structure is similar to other array.

For Example:-

```
struct student stu[100];
```

- Example:

```
struct family {  
    int member;  
    int children;  
    int number_of_male;  
    int number_of_female;  
    float total_Income;  
}f[3];
```

- In an array of structure all elements of the array are stored in adjacent memory locations. As we know each element of this array is a structure, and since all structure elements are always stored in adjacent locations.
- In above example, f[0]'s children, number_of_male, number_of_female and total_Income in memory would be immediately followed by f[1]'s children, number_of_male, number_of_female and total_Income and so on.

Program

- **WAP to store the information of ten students using structure.**
- a structure(student) contains three data members name, roll and marks.
- Then, an array of structure of 10 elements is created.
- Then, data(name, roll and marks) for students is asked to user and stored in array of structure. Then print the stored information.

Program

```
#include <stdio.h>

struct student{
    char name[50];
    int roll;
    float marks;
};

int main(){
    struct student s[10];
    int i;
    printf("Enter information of students:\n");
    for(i=0;i<3;++i)
    {
        s[i].roll=i+1;
        printf("\nFor roll number %d\n",s[i].roll);
        printf("Enter name: ");
        scanf("%s",s[i].name);
        printf("Enter marks: ");
        scanf("%f",&s[i].marks);
        printf("\n");
    }
    printf("Displaying information of students:\n\n");
    for(i=0;i<3;++i)
    {
        printf("\nInformation for roll number %d:\n",i+1);
        printf("Name: ");
        puts(s[i].name);
        printf("Marks: %.1f",s[i].marks);
    }
    return 0;
}
```

Output

```
Enter information of students:

For roll number 1
Enter name: Abc
Enter marks: 55

For roll number 2
Enter name: xy
Enter marks: 67

For roll number 3
Enter name: uy
Enter marks: 78

Displaying information of students:

Information for roll number 1:
Name: Abc
Marks: 55.0
Information for roll number 2:
Name: xy
Marks: 67.0
Information for roll number 3:
Name: uy
Marks: 78.0
```

Example:

The annual examination is conducted for 50 students for three subjects. WAP to read the data and determine the following:

- a) Total marks obtained by each student
- b) The highest marks in each subject and the Roll No of the student who secured it.
- c) The student who obtained the highest total marks

Expected Output:

```
Enter how many students: 3

Enter name and roll number for student 0 : A
101

Enter name and roll number for student 1 : B
102

Enter name and roll number for student 2 : C
103

Enter marks of student 0 for subject 0 : 57
Enter marks of student 0 for subject 1 : 67
Enter marks of student 0 for subject 2 : 78
Enter marks of student 1 for subject 0 : 89
Enter marks of student 1 for subject 1 : 90
Enter marks of student 1 for subject 2 : 56
Enter marks of student 2 for subject 0 : 65
Enter marks of student 2 for subject 1 : 45
Enter marks of student 2 for subject 2 : 34

Total marks obtained by student A are 202n
Total marks obtained by student B are 235n
Total marks obtained by student C are 144n
Student B got maximum marks = 89 in Subject : 0
Student B got maximum marks = 90 in Subject : 1
Student A got maximum marks = 78 in Subject : 2
B obtained the total highest marks.
```

```
#include<stdio.h>
#define SIZE 50
struct student {
    char name[30];
    int rollno;
    int sub[3];
};
void main() {
    int i, j, max, count, total, n, a[SIZE], ni;
    struct student st[SIZE];
    printf("Enter how many students: ");
    scanf("%d", &n);
    /* for loop to read the names and roll numbers*/
    for (i = 0; i < n; i++) {
        printf("\nEnter name and roll number for student %d : ", i);
        scanf("%s", &st[i].name);
        scanf("%d", &st[i].rollno);
    }
    /* for loop to read ith student's jth subject*/
    for (i = 0; i < n; i++) {
        for (j = 0; j <= 2; j++) {
            printf("\nEnter marks of student %d for subject %d : ", i, j);
            scanf("%d", &st[i].sub[j]);
        }
    }
}
```



```

/* (i) for loop to calculate total marks obtained by each student*/
for (i = 0; i < n; i++) {
    total = 0;
    for (j = 0; j < 3; j++) {
        total = total + st[i].sub[j];
    }
    printf("\nTotal marks obtained by student %s are %dn", st[i].name, total);
    a[i] = total;
}

/* (ii) for loop to list out the student's roll numbers who
    have secured the highest marks in each subject */
/* roll number who secured the highest marks */

for (j = 0; j < 3; j++) {
    max = 0;
    for (i = 0; i < n; i++) {
        if (max < st[i].sub[j]) {
            max = st[i].sub[j];
            ni = i;
        }
    }
    printf("\nStudent %s got maximum marks = %d in Subject : %d", st[ni].name, max, j);
}

```

Problems

1. An automobile company has serial number for engine parts starting from AA0 to FF9. The other characteristics of parts to be specified in a structure are: Year of manufacture, material and quantity manufactured.
 - (a) Specify a structure to store information corresponding to a part.
 - (b) Write a program to retrieve information on parts with serial numbers between BB1 and CC6.
2. There is a structure called **employee** that holds information like employee code, name, date of joining. Write a program to create an array of the structure and enter some data into it. Then ask the user to enter current date. Display the names of those employees whose tenure is 3 or more than 3 years according to the given current date.

Outline: Lecture 34 & 35

- Structure using function

• Passing struct to functions

Here's how you can pass structures to a function

```
1. #include <stdio.h>
2. struct student
3. {
4.     char name[50];
5.     int age;
6. };
7.
8. // function prototype
9. void display(struct student s);
10.
11. int main()
12. {
13.     struct student s1;
14.
15.     printf("Enter name: ");
16.     scanf("%[^\n]%*c", s1.name);
17.
18.     printf("Enter age: ");
19.     scanf("%d", &s1.age);
20.
21.     display(s1);    // passing struct as an argument
22.
23.     return 0;
24. }
```

```
25. void display(struct student s)
26. {
27.     printf("\nDisplaying information\n");
28.     printf("Name: %s", s.name);
29.     printf("\nAge: %d", s.age);
30. }
```

Output

```
Enter name: Bond
Enter age: 13

Displaying information
Name: Bond
Age: 13
```

Return struct from a function

Here's how you can return structure from a function:

```
1. #include <stdio.h>
2. struct student
3. {
4.     char name[50];
5.     int age;
6. };
7.
8. // function prototype
9. struct student getInformation();
10.
11. int main()
12. {
13.     struct student s;
14.
15.     s = getInformation();
16.
17.     printf("\nDisplaying information\n");
18.     printf("Name: %s", s.name);
19.     printf("\nRoll: %d", s.age);
20.
21.     return 0;
22. }
```

```
23. struct student getInformation()
24. {
25.     struct student s1;
26.
27.     printf("Enter name: ");
28.     scanf ("%[^\\n]%*c", s1.name);
29.
30.     printf("Enter age: ");
31.     scanf("%d", &s1.age);
32.
33.     return s1;
34. }
```

Here, the `getInformation()` function is called using

```
s = getInformation(); statement.
```

The function returns a structure of type `struct student`.

The returned structure is displayed from the `main()` function.

Notice that, the return type of `getInformation()` is also `struct student`.


```

#include <stdio.h>
struct student
{
    char name[50];
    int age;
};
//function prototype
void getInformation(struct student);
int main()
{
    struct student s;
    getInformation(s);
    printf("\nDisplaying information\n");
    printf("Name: %s", s.name);
    printf("\nRoll: %d", s.age);
    return 0;
}
void getInformation(struct student s1)
{
    printf("Enter name: ");
    gets(s1.name);
    printf("Enter age: ");
    scanf("%d", &s1.age);
}

```

Enter name: Ram

Enter age: 21

Displaying information

Name: 0■`

Roll: 44

Passing struct by reference

- We can also pass structs by reference (in a similar way like we pass variables of built-in type by reference).
- During pass by reference, the memory addresses of struct variables are passed to the function.

```

#include <stdio.h>
struct student
{
    char name[50];
    int age;
};
//function prototype
void getInformation(struct student *);
int main()
{
    struct student s;
    getInformation(&s);
    printf("\nDisplaying information\n");
    printf("Name: %s", s.name);
    printf("\nRoll: %d", s.age);
    return 0;
}
void getInformation(struct student *s1)
{
    printf("Enter name: ");
    gets(s1->name);
    printf("Enter age: ");
    scanf("%d", &s1->age);
}

```

```

Enter name: Ram
Enter age: 21

Displaying information
Name: Ram
Roll: 21

```

```

1. #include <stdio.h>
2. typedef struct Complex
3. {
4.     float real;
5.     float imag;
6. } complex;
7.
8. void addNumbers(complex c1, complex c2, complex *result);
9.
10. int main()
11. {
12.     complex c1, c2, result;
13.
14.     printf("For first number,\n");
15.     printf("Enter real part: ");
16.     scanf("%f", &c1.real);
17.     printf("Enter imaginary part: ");
18.     scanf("%f", &c1.imag);
19.
20.     printf("For second number, \n");
21.     printf("Enter real part: ");
22.     scanf("%f", &c2.real);
23.     printf("Enter imaginary part: ");
24.     scanf("%f", &c2.imag);
25.
26.     addNumbers(c1, c2, &result);
27.     printf("\nresult.real = %.1f\n", result.real);
28.     printf("result.imag = %.1f", result.imag);
29.
30.     return 0;

```

Output

```

For first number,
Enter real part: 1.1
Enter imaginary part: -2.4
For second number,
Enter real part: 3.4
Enter imaginary part: -3.2

result.real = 4.5
result.imag = -5.6

```

- In the above program, three structure variables `c1`, `c2` and the address of `result` is passed to the `addNumbers()` function. Here, `result` is passed by reference.
- When the `result` variable inside the `addNumbers()` is altered, the `result` variable inside the `main()` function is also altered accordingly.

Passing Structures to Function

```
struct student
{
char name[50];
int roll;
};
void display(struct student stu)
{
printf("Name: %s",stu.name);
printf("\nRoll: %d",stu.roll);
}
```

```
struct student update(struct student stu)
{
stu.name="hello";
return stu;
}
```

```
int main()
{
struct student stud;
scanf ("%s", stud.name);
scanf("%d", &stud.roll);
display(stud);
stud= update(stud);
display(stud);
return 0;
}
```

Passing Array of Structures to Function

struct Example

```
{  
int num1;  
int num2;  
}
```

void accept(**struct** Example sptr[],**int** n)

```
{  
int i;  
for(i=0;i<n;i++)  
{  
printf("\nEnter num1 : ");  
scanf("%d",&sptr[i].num1);  
printf("\nEnter num2 : ");  
scanf("%d",&sptr[i].num2);  
}  
}
```

void print(**struct** Example sptr[],**int** n)

```
{  
int i;  
for(i=0;i<n;i++)  
{  
printf("\nNum1 : %d",sptr[i].num1);  
printf("\nNum2 : %d",sptr[i].num2);  
}}  

```

void main()

```
{  
int i;  
struct Example s[3];  
accept(s,3);  
print(s,3);  
}
```

References

- <https://www.tutorialspoint.com/cprogramming/index.htm>
- H. Schildt, “C: The Complete Reference”, Tata McGraw-Hill Education, 4th Edition, TMH 2000
- Yashwant Kanetkar, “Let us C”, Vth Edition, Bpb publications